Architettura Software: Gestione Cluster GPU

Team di Sviluppo - Ingegneria del Software

February 8, 2026

1 Panoramica del Sistema

Il progetto implementa un sistema per la gestione condivisa di risorse di calcolo (GPU). L'architettura segue il pattern **Layered Architecture**, garantendo una netta separazione tra la persistenza dei dati, la logica di business e l'interfaccia utente.

2 Struttura del Progetto (Schema)

Di seguito è riportata la struttura delle directory e dei file Java ('src/main/java/it/unifi/ing/').

```
src/main/java/it/unifi/ing/
├── domain (IL "COSA" - Dati Passivi)
│   ├── Utente.java ..... Abstract: id, nome, email, password
│   ├── Developer.java ..... extends Utente
│   ├── ModelProvider.java ..... extends Utente
│   ├── Supervisor.java ..... extends Utente
│   │
│   ├── Wallet.java ..... Saldo, storico transazioni
│   ├── Modello.java ..... Nome, costo, file, StatoModello
│   ├── Sessione.java ..... Utente, Modello, GPU, timestamp
│   │
│   ├── GPU.java [Subject] ..... Dati: id, temp, stato
│   └── ClusterGPU.java [Singleton] ..... Gestore HW live
│
├── dao (IL "MAGAZZINO" - Persistenza Fake)
│   ├── interfaces
│   │   ├── UtenteDAO.java
│   │   ├── ModelloDAO.java
│   │   ├── SessioneDAO.java
│   │   └── GpuDAO.java <- NUOVO
│   └── memory [Repository]
│       ├── InMemoryUtenteDAO.java ..... Map<Integer, Utente>
│       ├── InMemoryModelloDAO.java ..... Map<Integer, Modello>
│       ├── InMemorySessioneDAO.java ..... Map<Integer, Sessione>
│       └── InMemoryGpuDAO.java <- NUOVO ..... Map<Integer, GPU>
│
└── business (IL "CERVELLO" - Logica Attiva)
    ├── Timer.java [Singleton] ..... Attore di Sistema
    │
    └── services (Logica Pura)
        ├── AuthService.java <- Login/Register
        ├── LoadBalancerService.java [Observer]
        └── BillingService.java ..... Addebito costi
```

```
│
│
│
│
│
├──
└── controllers (Logica Interazione)
    ├── AuthController.java <- Menu Iniziale
    ├── GestioneModelliController.java
    ├── VerificaController.java
    ├── SessioneController.java
    └── GestioneUtenteController.java

├──
└── ■ view (L'INTERFACCIA)
    └── Main.java ..................................................... Entry point
```

3 Descrizione Architettura e Flusso Dati

3.1 1. L'Avvio (Bootstrapping) - Main.java

Il `Main` funge da configuratore iniziale dell'applicazione:

1. **Istanzia i DAO in memoria:** Vengono create le mappe per la persistenza di Utenti, Modelli, Sessioni e GPU.
2. **Inizializza il ClusterGPU:** Il Singleton `ClusterGPU` recupera la lista delle schede hardware disponibili chiamando il `GpuDAO`.
3. **Avvia il Timer:** Viene lanciato il thread di background per la gestione periodica (billing e monitoraggio).
4. **Lancia AuthController:** Viene mostrato il menu di accesso all'utente.

3.2 2. Accesso e Sicurezza - Auth

La gestione dell'accesso è separata in due componenti per pulizia architetturale:

- **AuthController:** Gestisce l'interazione con l'utente (CLI), richiedendo input per Login o Registrazione.
- **AuthService:** Contiene la logica di business.
  - *Registrazione:* Verifica l'unicità dell'email, crea l'oggetto `Utente` appropriato e lo salva tramite `UtenteDAO`.
  - *Login:* Verifica le credenziali e restituisce l'utente loggato.

3.3 3. Il "Cuore": Gestione Hardware e Sessioni

- **Richiesta Risorse:** Quando un Developer avvia una sessione, il `SessioneController` interroga il `ClusterGPU`.
- **Singleton:** `ClusterGPU` è l'unico punto di accesso all'hardware. Assegna una GPU libera recuperata dalla lista caricata all'avvio.
- **Sessione:** Viene istanziato un oggetto `Sessione` che collega l'Utente, il Modello scelto e la GPU fisica.

3.4 4. Monitoraggio Live (Observer Pattern)

Il sistema simula il comportamento reale dell'hardware tramite il pattern Observer:

- **Subject (GPU):** Ogni istanza di GPU monitora i propri dati (temperatura). Se la temperatura supera i 90°C, notifica gli osservatori.
- **Observer (LoadBalancerService):** Questo servizio è in ascolto. Alla ricezione di un allarme surriscaldamento, interviene spegnendo la GPU o terminando la sessione per sicurezza.

3.5 5. Il Ciclo Economico (Background Threads)

- **Timer:** Un attore di sistema che scandisce il tempo.
- **BillingService:** Ad ogni intervallo (tick), calcola il costo della sessione corrente (costo modello + costo orario GPU) e scala i token dal `Wallet` dell'utente tramite `UtenteDAO`.

4 Design Pattern Utilizzati

Singleton Utilizzato in `ClusterGPU` e `Timer` per garantire un'unica istanza centrale di controllo.

Observer Implementato tra `GPU` (Subject) e `LoadBalancerService` (Observer) per la gestione asincrona degli eventi critici.

Repository/DAO Le interfacce DAO disaccoppiano la logica di business dalla persistenza dei dati (attualmente in memoria).